Failure and resilience in Pastry

COP6539 Project 3, bonus · Ahmed Rageeb Ahsan, Saiful Islam

The main project runs on a network where nothing ever fails. This report adds the three failures the assignment names, implements the recovery mechanisms the Pastry paper describes, and measures which mechanism defends against which failure. Every experiment uses 1,000 nodes making three lookups each, repeated three times; each point is the mean.

How resilience was tested

The failure models. All three strike after the network is built and its objects stored, so what is measured is whether lookups survive a failing system. Each is driven by a single parameter.

- `kill=p` — **a node dies.** A fraction *p* of peers is killed outright, with no warning to anyone, as the lookups begin. Anything in flight to them is lost.
- `drop=q` — **a connection dies temporarily.** Each transmission is lost independently with probability *q*; the same link may well work on the next attempt.
- `links=r` — **a connection dies permanently.** A fraction *r* of node pairs cannot reach each other for the whole run. Whether a link is dead is a pure function of its two endpoints, so every node agrees on it without any coordination, and retrying it never succeeds.

The defences. Each follows the Pastry paper (section 2.5) or PAST, the storage system built on it, and each can be switched on independently so its contribution can be measured on its own.

- **Failure detection** — every node monitors the nodes in its leaf set and routing table. An Erlang monitor is the runtime's version of the keep-alive messages Pastry's leaf-set members exchange.
- **Repair** — on a leaf-set failure, ask the farthest live leaf on that side for its leaf set; on a routing-table failure, ask another entry in the same row for its entry. Both are the paper's own procedures.
- **Replication** — each object is stored on the four nodes closest to its key, as PAST does, so it outlives the death of any one of them.
- **Retry** — every lookup carries a timeout; a lookup that times out or reaches a node without the object is retried, up to three attempts in all, along either the same route or a randomised one.

Checking the checks. Every request is counted exactly once: found, not found, or given up on after its last timeout. A late reply to an attempt already retried is recognised by its reference and ignored, so no request can be counted twice. Randomised routing was verified separately against a brute-force oracle and reaches the true closest node every time.

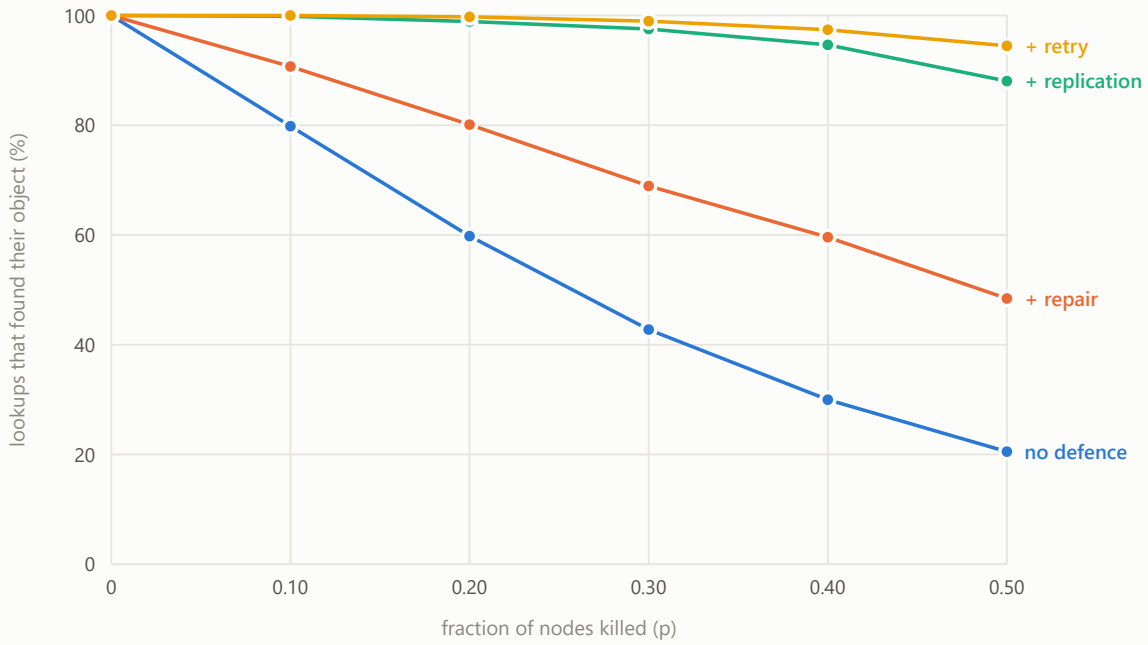
---

1. A node dies

no defence + repair + replication + retry

Node death

Defences added one at a time, each line including all the ones before it.



Repair fixes routing; it cannot bring back data. At p = 0.2 lookups succeed 59.8% of the time with no defence, and repair raises that to 80.1%. What it leaves behind is telling: the remaining loss equals the fraction of nodes killed, at every value of p (table below). Those are objects whose only copy lived on a dead node. Once routing is repaired, a lookup arrives at exactly the right place and finds nothing there — and no improvement to routing can change that.

Replication is what saves the data. It lifts p = 0.2 to 98.9% and still holds 88.0% when half the network dies. Retry then adds more than might be expected, up to 94.5% at p = 0.5, by recovering lookups that set off in the brief window before repair completed.

With every defence on, the system reaches its theoretical floor. An object stored on four nodes is lost only if all four die, which happens with probability p^4. The fully defended curve sits on $1 - p^4$ almost exactly: the only lookups that still fail are for objects no mechanism could possibly have saved. Likewise repair alone sits on $1 - p$, the survival rate of a single copy.

Nodes killed (p)	Repair only	1 − p	Every defence	$1 - p^4$
0.1	90.7%	90.0%	100.0%	99.99%
0.2	80.1%	80.0%	99.8%	99.84%
0.3	68.9%	70.0%	99.0%	99.19%
0.4	59.6%	60.0%	97.4%	97.44%
0.5	48.4%	50.0%	94.5%	93.75%

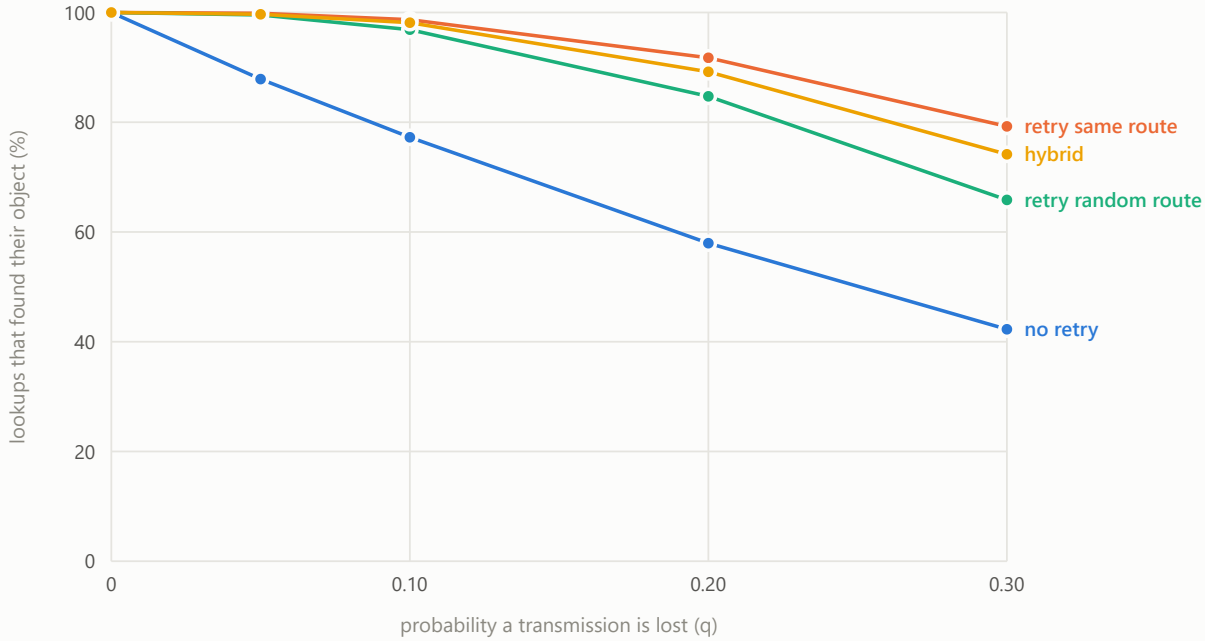
The two mechanisms answer different questions — repair asks *can the message get there*, replication asks *is anything there to find* — and surviving node death needs both.

2. A connection dies temporarily

no retry retry same route retry random route hybrid

Temporary connection failure

Each transmission independently lost with probability q.



A lookup crosses about 2.48 links and losing any one loses it, so it should survive with probability $(1 - q)^{2.48}$. At q = 0.2 that predicts 57.4%; measured, 58.0%.

Retrying fixes most of it, and here **retrying the same route is best** — at every value of q: 91.7% against 84.7% for a random route at q = 0.2. Each attempt along the same short route is an independent draw, so three attempts should leave only $(1 - 0.574)^3$ of lookups failing, a predicted 92.3%. A randomised route is slightly longer, and every extra hop is another chance to be lost.

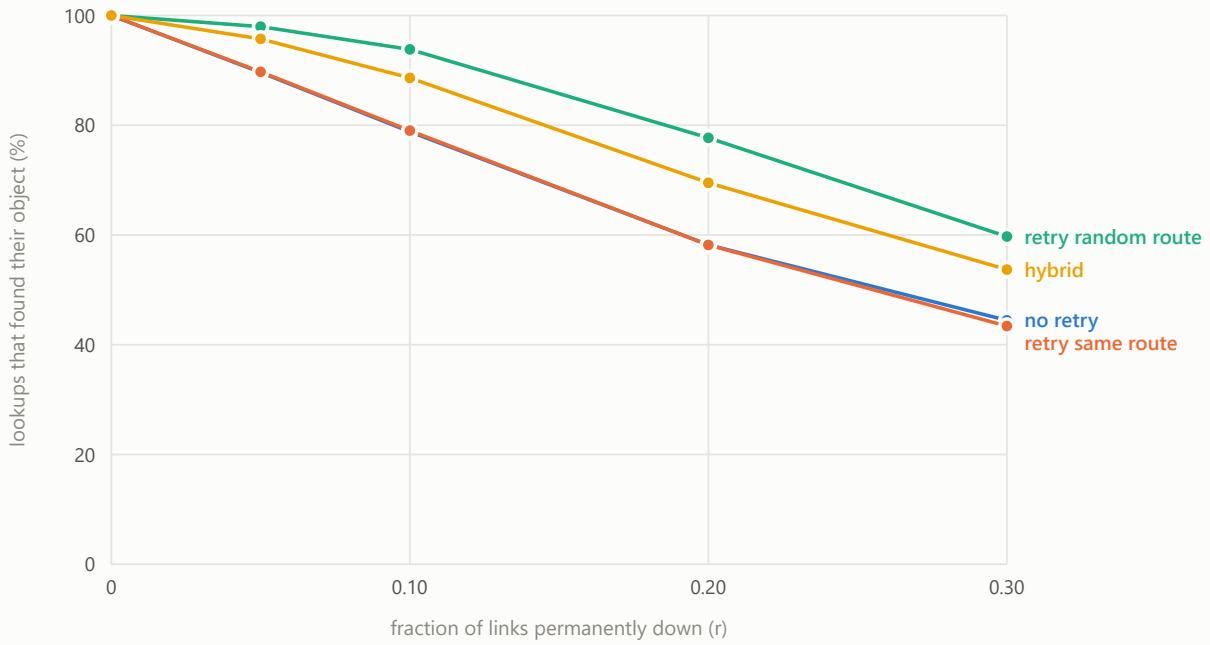
---

3. A connection dies permanently

no retry retry same route retry random route hybrid

Permanent connection failure

A fraction r of node pairs cannot communicate for the whole run.



Here the ranking **inverts**. Retrying the same route is worthless: 58.2% at r = 0.2, against 58.2% with no retry at all — the two lines lie on top of each other at every value of r. Pastry's routing is deterministic, so the same lookup from the same node takes the same path, straight back into the same dead link, and three attempts fail exactly as one did. Only a **randomised** route escapes it, reaching 77.7%. This is why the Pastry paper suggests randomising retries.

Findings

> **No single retry policy is best, and a node cannot tell which one it needs.** The same-route retry that wins against temporary failure is the one that fails against permanent failure, and the reverse. A node whose lookup times out cannot tell which kind of failure lost it.
>
> The obvious compromise — retry the same route once, then randomise — was tested as *hybrid*, and it is never the best at anything. It lies strictly between the two pure policies at every one of the eight failure levels tested; at 20% it reaches 89.2% against temporary loss and 69.5% against permanent loss. With three attempts, spending one on the same route is one attempt wasted whenever the failure is permanent.
>
> So the simulator defaults to the randomised retry, the policy with the best worst case. It gives up a little against temporary failure to avoid failing badly against permanent failure — the right trade when the kind of failure is unknown.

Every failure mode needed its own defence. Node death was survivable only with replication; temporary loss only with retry; permanent link loss only with *randomised* retry. Pastry's resilience is not one property but several separate mechanisms, each covering one way of failing.

Reproducing

```
project3 1000 3 kill=0.2
project3 1000 3 kill=0.2 resilient
project3 1000 3 drop=0.2 attempts=3 retry=det
project3 1000 3 links=0.2 attempts=3 retry=rand
```